ClosingScript

This project may appeal to programmers who dislike the significant indentation in CoffeeScript but otherwise like the language for its simplicity and expressiveness. ***ClosingScript*** addresses this sensitive problem by adding closing statements to CoffeeScript syntax to close multi-line statements, making whitespace no longer significant.

ClosingScript is a text preprocessor, not a fork, that regenerates significant indentation using the closing statements.

A closing statement consists of the slash followed by the name of the statement that opens the multi-line statement.

List of closing keywords:
/**`if`**, **`/unless`**, **`/while`**, **`/until`**, **`/for`**, **`/switch`**, **`/loop`**, **`/try`**, **`/class`**, **`/end`** (function)

Advantages:

- Whitespace is not significant. No block ambiguity.
- Clear closure of statements.
- Safe copy-pasting.
- Safe auto-formatting.

Disadvantages:

- Slightly more verbose syntax.

Usage:
It is assumed the CoffeeScript compiler is installed. Compiler errors match ***ClosingScript*** sources. The preprocessor outputs to *stdout.*

`closing` *`script.coffee`* `|` **`coffee`** `-s -c >` *`script.js`*
Convert and compile to JavaScript in one step.

`closing` *`script.coffee`* `>` *`script.converted.coffee`*
Get converted copy of source.

`closing -f` *`script.coffee`* `>` *`script.formatted.coffee`*
Get formatted copy of source. Code is only reindented consistently.

Multi-line statements

The syntax of a multi-line statement indentical to standard CoffeeScript except for the addition of a closing keyword as the last line. The list below is not exhaustive but gives a good idea of how closing keywords are added.

functionName = (parameters) **->** | **=>**
  code
/end [optional following code] # examples for following code: ***/end,*** *timeout* or ***/end)***

do ->
  code
/end

[variable =] **if** | **unless** condition …
  code
[**else if** | **unless** condition
  code]
[**else**
  code]
/if

[variable =] **for** iterable [guard clauses]
  code
/for

[variable =] **while** | **until** condition [guard clauses]
  code
/while

[variable =] **loop**
  code
/loop

[variable =] **switch** [variable]
 when
  code
 [**when** condition **then** *code*]
 [**else**
  code]
/switch

class *name*
  code
/class

try
  code
[**catch** …
  code]
[**finally** …
  code]
/try

Single-line statements (one-liners)

The syntax of a single-line statement is identical to standard CoffeeScript. A closing keyword is not needed. The preprocessor assumes it is a single-line statement when it detects the keyword **then**.

[variable =] **if | unless** condition **then** *code* [**else** *code*]

[variable =] **for** iterable [guard clauses] **then** *code*

[variable =] **while | until** condition [guard clauses] **then** *code*

Geany code editor

In Linux, this command can be added to the Build menu of the Geany code editor. Its converts and compiles in one step, with errors falling back at guilty lines in the editor window, as well as error code propagation from the preprocessor and the compiler.

set -o pipefail; **closing** "%f" | **coffee** -s -c > "%e".js 2> >(**sed** 's/\[stdin\]/%f/' >&2)